Arquitectura del Backend

Sistema de Gestión de Gastos. Una API REST modular y escalable construida con TypeScript, Node.js y principios de diseño avanzados.

Visión General y Tecnologías

Stack Tecnológico Principal

- **TypeScript:** Tipado estático robusto y seguro.
- **Express.js:** Framework HTTP para rutas y middlewares.
- **MySQL / JSON:** Base de datos relacional y alternativa local ágil.
- **Zod:** Validación estricta de datos de entrada (runtime).
- **JWT & bcrypt:** Autenticación segura y hash de contraseñas.

Principios de Diseño

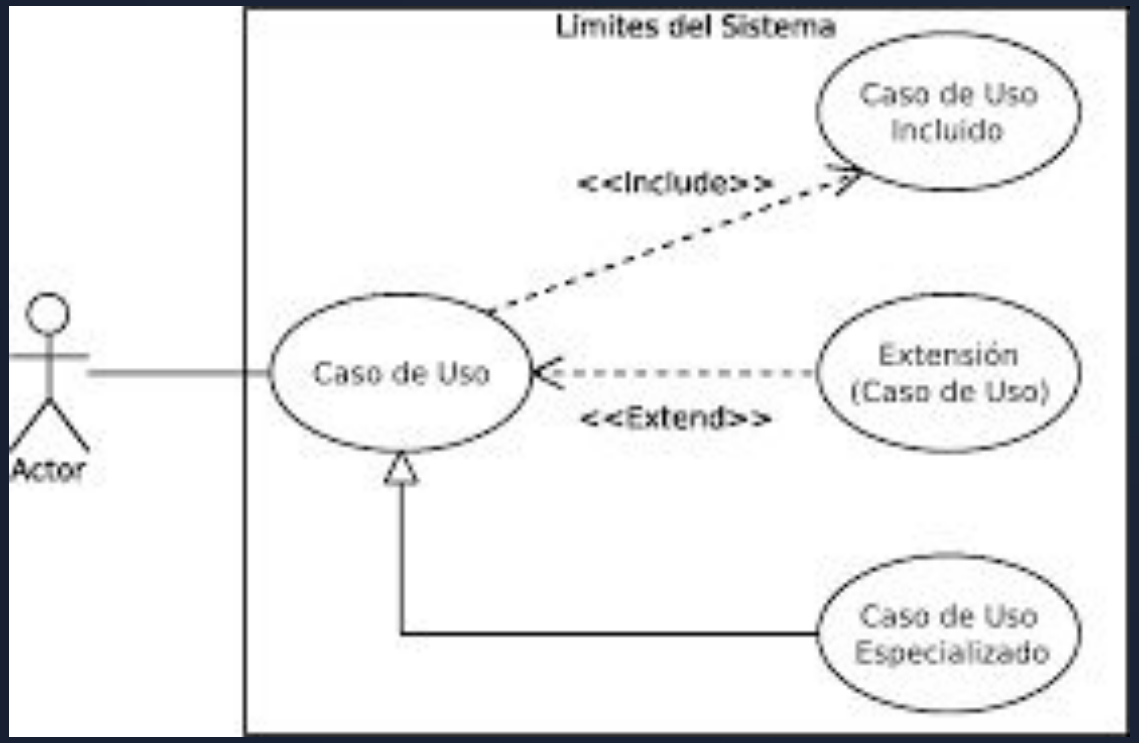
- **Separación de Responsabilidades:** Cada capa (Repo, Service, Controller) tiene un único propósito.
- **Inversión de Dependencias:** Servicios dependen de interfaces (contratos), no de bases de datos concretas.
- **Principio Abierto/Cerrado:** Patrones Observer y Strategy permiten extender el código sin alterarlo.

Flujo de una Petición

El Viaje del Dato

Cuando el frontend envía un POST a /api/expenses, ocurre una secuencia orquestada:

- **Middleware (Express):** Evalúa CORS y parsea el JSON entrante.
- **authMiddleware:** Verifica la existencia y validez del token JWT.
- **Controller:** Valida el payload con Zod. Si falla, retorna error 400. Si aprueba, delega la tarea.
- **Service:** Aplica lógica de negocio, persiste vía Repository y emite eventos al Observer.
- **Respuesta:** El cliente recibe un JSON estandarizado de éxito o error gestionado por el errorHandler.



Capa 1 & 2: Modelos y Schemas

Capa 1: Modelos (TypeScript)

Definen la forma exacta de los datos. Actúan durante la **compilación** para proteger al desarrollador.

- IExpense / IUser: Las entidades completas que viven en la base de datos (con ID y fechas).
- **DTOs (Data Transfer Objects):** Versiones recortadas para casos específicos. Ej: CreateExpenseDTO no requiere un ID porque el backend lo genera.

Capa 2: Schemas (Zod)

Garantizan la integridad en **tiempo de ejecución**. Protegen al servidor de payloads maliciosos o erróneos.

- createExpenseSchema: Exige que el monto sea positivo, el título tenga máximo 100 caracteres y la fecha sea válida.
- Fallo automático: Zod intercepta el dato; si es inválido, rechaza la petición con un 400 Bad Request antes de tocar la base de datos.

Capa 3: Repositorios

Contratos (Interfaces)

Define **qué** se puede hacer, aislando la lógica de negocio del almacenamiento. IExpenseRepository dicta que deben existir métodos como create() o findAll(), sin importar el motor subyacente.

Implementación MySQL

Conexión real utilizando un pool de conexiones para alta concurrencia. Contiene auto-migración (crea tablas si no existen) y mapea resultados snake_case a propiedades camelCase nativas de JS.

Implementación JSON

Una alternativa ligera que persiste datos en archivos .json locales. Ideal para entornos de desarrollo rápido o demostraciones, implementando exactamente la misma interfaz que MySQL.

Capa 4 & 5: Servicios y Controladores

Capa 4: Servicios (El Cerebro)

Contiene la **lógica de negocio pura**. Sin conocimientos de HTTP.

- AuthService: Hashea contraseñas, verifica credenciales y emite tokens JWT. Cifra datos sensibles antes de enviarlos al exterior.
- ExpenseService: Aísla la información por usuario, asegurando que nadie edite gastos ajenos. Orquesta acciones entre Repositorios y Observers.

Capa 5: Controladores (El Tráfico)

Traducen peticiones web a llamadas internas. **Sin lógica de negocio.**

- Extracción: Toman req.body y req.user.
- Validación y Delegación: Ejecutan schema.safeParse() y llaman a métodos del Service.
- Estandarización: Responden siempre con estructuras JSON coherentes: { status: 'success', data }.

Red de Seguridad: Middlewares, Errores y Rutas

Capa 6: Middlewares

Los "guardaespaldas". authMiddleware decodifica tokens; si es inválido, aborta con error 401. Extiende el request de Express inyectando el req.user validado para toda la cadena.

Capa 7: Errores Personalizados

El errorHandler global atrapa cualquier excepción. Identifica si es un AppError (planeado, devuelve el código HTTP asignado) o un bug de sistema (retorna un genérico 500 ocultando detalles técnicos).

Capa 8: Rutas

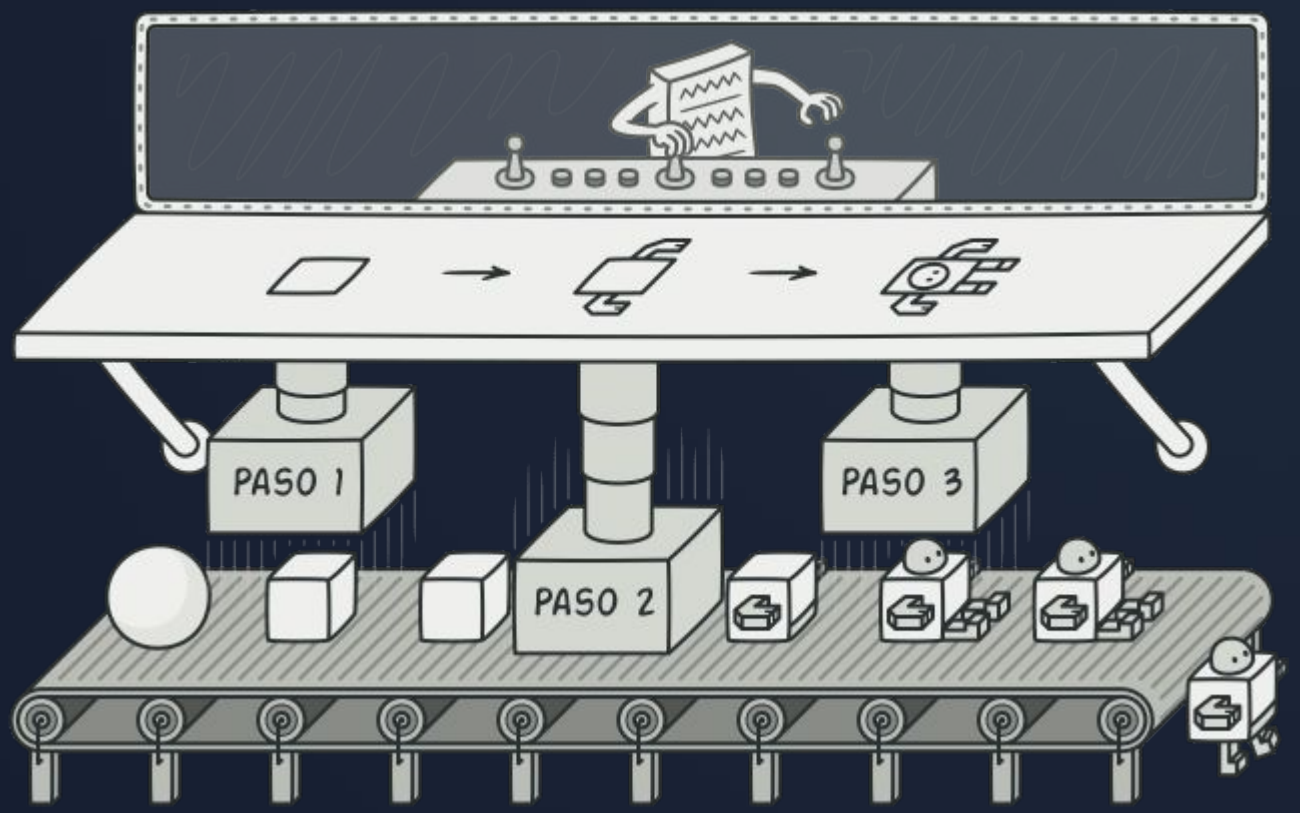
Mapean URLs a Controladores. Rutas como /api/auth/login son públicas, mientras que /api/expenses están agrupadas bajo un router.use(authMiddleware) para protección masiva eficiente.

Patrones: Creacionales y Estructurales

Optimización e Inyección

Los Patrones de Diseño resuelven problemas complejos estandarizando el código.

- **Factory (Creacional):** RepositoryFactory lee .env (DB_TYPE) y decide instanciar MySQL o JSON. Permite cambiar el motor de base de datos alterando una sola línea de configuración, sin tocar el Service.
- **Proxy (Estructural):** CacheExpenseProxy envuelve al repositorio real. Guarda respuestas en memoria RAM. Si un usuario pide sus gastos mensuales, el Proxy intercepta y entrega datos cacheados en ~1ms, evitando consultas costosas en DB (~80ms). Se invalida al crear o editar gastos.



Patrones: Comportamiento

Observer (Notificaciones Desacopladas)

Reacciona a eventos sin meter la lógica dentro del servicio principal.

- El ExpenseSubject mantiene un array de suscripciones.
- Al registrar un gasto alto, se llama a notify().
- Inmediatamente, HighPriorityAlertObserver y WhatsAppObserver procesan el envío de alertas. Agregar un nuevo canal (ej. Email) solo requiere crear una clase nueva, respetando el Principio Abierto/Cerrado.

Strategy (Filtros Dinámicos)

Elimina gigantescas cadenas de if/else en el controlador.

- ExpenseFilterContext posee una "ranura" de estrategia.
- Se pueden enchufar clases como FilterByMonthStrategy o FilterByPriorityStrategy.
- El contexto simplemente llama a strategy.execute(expenses), intercambiando lógicas complejas de filtrado en tiempo de ejecución.

Seguridad y Autenticación

- **Criptografía bcrypt:** Las contraseñas se almacenan procesadas bajo 12 rondas de hashing salino. Jamás en texto plano.
- **Tokens JWT:** Generados con firmas secretas impenetrables. Proveen estados sin estado (stateless) y caducan obligatoriamente tras 7 días de inactividad.
- **Sanitización Omit:** Función dedicada toProfile() extirpa cualquier rastro del password hash antes de devolver el perfil de usuario a las peticiones del cliente.
- **Aislamiento Inviolable:** Todo query incluye estrictamente un userId mandatorio evaluado desde el JWT, aislando entornos multi-tenant.
- **Mensajes Genéricos:** Durante un login fallido, las respuestas no confirman la existencia de correos, mitigando ataques de enumeración de usuarios.

Calidad y Optimización

Testing Unitario (Jest)

Las pruebas aseguran la lógica de negocio simulando el entorno real.

- **Mocks (Dobles):** Funciones espía (jest.fn()) emulan la base de datos, testeando la aplicación sin necesidad de levantar infraestructura de red externa.
- **Patrón AAA:** Pruebas estructuradas en Arrange (preparar datos), Act (invocar función) y Assert (verificar respuestas o llamadas delegadas).

Estructuras de Datos Nativas

Selección cuidadosa para maximizar la eficiencia y reducir complejidad algorítmica.

- **Map (Caché):** El proxy implementa un Map JS, asegurando complejidad O(1) con llaves compuestas tipo "userId-año-mes" para accesos directos de lectura.
- **Arrays Dinámicos:** Los observadores se apilan en memoria lineal facilitando su iteración con forEach en O(n) durante la notificación masiva.

Ensamblaje Final (server.ts)

1. Setup Base

La aplicación Express inicializa. Se configuran CORS y parsers para habilitar la deserialización JSON y cabeceras de seguridad cruzada globales.

2. Inyección de Repos

El RepositoryFactory inspecciona el entorno e inyecta dinámicamente conectores MySQL o JSON, los cuales a su vez son envueltos por Proxies de caché de RAM.

3. Core y Eventos

Los Servicios son materializados pasándoles por constructor sus repositorios correspondientes. En paralelo, los sistemas de Observer son ensamblados a los canales de alerta.

4. Endpoints Finales

Controladores se enlazan con enrutadores, empacados bajo la raíz /api. Se arranca Swagger para la documentación interactiva y el capturador de errores maestro se sitúa en la cola final.